

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Due date: 2025.12.30 23:59:59

Important Notice – Use of AI Tools

In this assignment, you must use at least one AI assistant (e.g. ChatGPT, Gemini, Claude, Grok, M365 Copilot) as a learning tool to help you:

- review definitions,
- compare tree variants, and
- organize your report.

You are not allowed to let the AI directly produce your final diagrams or final report content without your own understanding and rewriting.

You must log all AI prompts and services used (see “AI Usage Log” section below).

1. Goal of This Assignment

In the lectures, we introduced the concept of the tree as a data structure, starting from the general tree and then moving to more specialized forms.

In this assignment, you will:

- Understand and clearly define:
 - General tree
 - Binary tree
 - Complete binary tree
 - Binary search tree (BST)
 - AVL tree
 - Red-Black tree
 - Max heap
 - Min heap
- Build a hierarchy and transformation path from the general tree to these variants, and explain how each variant adds more structure or constraints.
- Use a fixed list of integers to construct multiple tree variants and visualize them.
- Choose one real-world application for each tree type and explain why that data structure fits the application.
- Practice using AI tools as study companions and keep a simple Q&A log.

2. Given Data

Use the following 20 integers as the input data for all your tree constructions:

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

You will reuse this same sequence for every tree type (binary tree, complete binary tree, BST, AVL, Red-Black, max heap, min heap).

3. Deliverables

Please complete your work in the Student Worksheet Companion and upload it to the YZU Portal System.

Your report should include the following parts:

a. Definitions (Concept Review)

Provide clear, concise definitions for each of the following:

1. General tree
2. Binary tree
3. Complete binary tree
4. Binary search tree (BST)
5. AVL tree
6. Red-Black tree
7. Max heap
8. Min heap

You are encouraged to use AI tools to help you understand these concepts, but you must rewrite the definitions in your own words.

b. Hierarchy and Transformation of Tree Variants

Based on the definitions above, build a “tree family hierarchy” that shows how these structures are related. For example:

- General tree → Binary tree
- Binary tree → Complete binary tree / Binary search tree
- BST → AVL tree / Red-Black tree
- Binary tree → Max heap / Min heap

Tasks:

- Draw a diagram or flow chart that shows the transformation or specialization path: general tree → binary tree → complete binary tree → BST → AVL/Red-Black, etc.
- For each arrow (transformation), briefly explain:
 - What new constraint or property is added?
 - e.g., “Binary tree = tree with at most 2 children per node”,
“BST = binary tree with left < root < right”,
“AVL = BST with strict height-balance rule”, etc.

c. Tree Construction with the Given Integers

Using the given 20 integers, construct the following tree variants:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)

4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Important Hint / Restriction:

- For these trees, you must use tree visualization tools (e.g., online visualizers or software) to build and display the tree.
- You may not ask AI tools to directly generate the final tree pictures for you.
- Instead:
 - Use AI only to help you understand algorithms,
 - Then apply those algorithms in a visualizer (or your own implementation).

What to submit for this part:

For each tree type:

- A snapshot (image) of the constructed tree.
- The URL / name of the visualization tool you used.
- A short note on how you inserted the integers (e.g., “insert in the given order as BST”, “build max heap using heapify”, etc.).

d. Application Example for Each Tree

For each of the following:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)
4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Choose one application (real-world or system-level) and explain:

1. Application description
 - e.g., priority scheduling, dictionary lookup, memory allocation, database indexing, etc.
2. Why this tree structure fits
 - What property of this data structure makes it suitable?
 - Example:
 - Max heap → good for priority queue because the largest element is always at the root, so extracting max is efficient.
 - Red-Black tree → good for standard library maps/sets because it guarantees $O(\log n)$ operations even under many insertions/deletions.

Your explanation should show that you understand the link between the data structure and its use case.

e. Report Layout and Organization

You are free to design the layout of your report, but it should:

- Be well-structured (use sections, headings, tables, and diagrams).
- Have a clear flow from:
 - definitions →
 - hierarchy/transformation →
 - constructed trees →
 - applications →
 - AI usage log.
- Be easy for another student to read and learn from.

Feel free to use AI to suggest a good outline, but you must decide and finalize the layout yourself.

f. AI Usage Log (Q&A Table)

Every time you use an AI copilot service for this assignment, record:

- Index (1, 2, 3, ...)
- Prompt (what you asked)
- Service (e.g., ChatGPT, Gemini, Copilot, ...)

Example log table:

Index	Prompt	Service
1	Assist me to have the definition of general tree, binary tree, complete binary tree, binary search tree, AVL tree, red-black tree, max heap and min heap for self-learning.	ChatGPT
2	Explain the difference between AVL tree and Red-Black tree in terms of balancing strategy and use cases.	Gemini
...	...	...

Place this table at the end of your report.

4. Evaluation (100 pts)

A possible breakdown (you can adjust if needed):

- a. Concept definitions (20 pts)
 - Correctness and clarity of all 8 tree type definitions.
- b. Hierarchy & transformation explanation (20 pts)
 - Clear diagram / explanation of how each tree variant evolves from the general tree.
 - Correct identification of constraints/invariants.
- c. Tree constructions & visualizations (25 pts)
 - Correct constructions for each tree type using the given integers.
 - Proper screenshots and tool URLs.

- Consistent insertion / heap-building strategy descriptions.
- d. Applications & explanations (20 pts)
 - One application per tree type.
 - Clear explanation linking data structure properties to the application.
- e. Report organization & AI usage log (15 pts)
 - Logical report structure and readability.
 - AI log completeness (all prompts listed with service names).
 - Thoughtful use of AI as a learning assistant, not as a copy-paste generator.

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student Worksheet Companion

Due date: 2025.12.30 23:59:59

Academic Integrity and AI Usage Statement

In this assignment, you must use AI tools (such as ChatGPT, Gemini, Claude, Grok, M365 Copilot, etc.) as learning assistants, but you must also take full responsibility for understanding and organizing your own work.

1. Permitted Use of AI Tools

You may use AI to:

- Review or clarify definitions and concepts.
- Compare different tree data structures.
- Get suggestions for report layout or examples.
- Ask for explanations of algorithms (e.g., BST insertion, AVL rotation, heapify process).

You should read, think about, and rewrite the content in your own words.

2. Not Permitted

- Do not copy/paste AI-generated content directly as your final answer.
- Do not ask AI to draw the final diagrams or directly produce the final tree screenshots.
- Do not ask AI to complete the whole assignment report for you.

3. Your Responsibility

- You are responsible for understanding the definitions and algorithms.
- You are responsible for verifying whether AI answers are correct or not.
- You must produce your own original explanations and diagrams.

4. AI Usage Log

- You must record all AI queries related to this assignment.
- At the end of your report, include an AI Usage Log table with: Index, Prompt, AI service name.

By submitting this assignment, you acknowledge that you have used AI tools only as study aids, and that the final content of this assignment represents your own understanding and work.

Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

1. General Tree

Definition:一種階層式的資料結構，由有限個節點組成並具有一個根節點。除了根之外，

每個節點僅有一個父節點，而其餘節點可再分成多個互不重疊的子樹，因此可以用遞迴方式定義：一棵 **General Tree** 由一個 **root** 以及零個或多個子樹組成。**General Tree** 是連通且無循環的結構，每個節點可以擁有任意數量的子節點且子節點之間一般沒有強制的順序。

2. Binary Tree

Definition: 是一種樹狀結構，具有唯一的根節點，且每個節點**最多只能有兩個子節點**，並且分別稱為左子節點與右子節點。以遞迴方式定義：一棵二元樹要不是空樹，就是由一個根節點，以及一棵左子樹與一棵右子樹(兩者本身也是二元樹)所組成。且 **Binary Tree** 也延續著 **General Tree** 連通且無循環的定義。

3. Complete Binary Tree

Definition: 一種特殊的 **Binary Tree** 結構，除了要滿足原本 **Binary Tree** 的定義，還會要求樹的節點必須儘量緊密排列，除了最後一層之外，所有層都必須被完全填滿，而最後一層的節點必須**由左向右連續排列**，不能出現左邊有空位卻在右邊仍有節點的情況。

4. Binary Search Tree (BST)

Definition: 是一種具排序規則的 **Binary Tree**，除了滿足原本 **Binary Tree** 的規則，還會要求:對於任意一個節點，其左子樹中的所有節點值都必須**小於**該節點，而右子樹中的所有節點值則必須**大於**該節點，且這個規則會在每個子樹上遞迴成立(兩個子樹也會是 **BST**)。

5. AVL Tree

Definition: 一種**自動保持平衡的 **BST****，除了遵守 **BST** 的基本規則之外，它還加入了嚴格的平衡條件: 對於樹中任何一個節點，其左、右子樹的高度差不得超過 **1**，當插入或刪除節點導致樹失去平衡時，**AVL Tree** 會透過一系列 **rotations** 自動調整回接近平衡的狀態。由於平衡程度高，**AVL Tree** 通常能維持穩定的 $O(\log n)$ 搜尋效能。

6. Red-Black Tree

Definition: 另一種**自動保持平衡的 **BST****，除了遵守 **BST** 的基本規則之外，他是使用**顏色規則**來維持平衡。每個節點都被標記為紅色或黑色，並必須滿足以下核心規則：根節點為黑色、且把不存在的子節點(**NULL**)視為**黑色的虛擬葉節點(**NIL**)**，並要求從任一節點到其所有 **NIL** 節點的路徑都擁有相同數量的黑色節點。這些規則確保樹不會嚴重失衡，使搜尋、插入與刪除皆能維持 $O(\log n)$ 。當結構被破壞時，紅黑樹會透過重新著色與 **rotations** 來恢復規則與平衡。

7. Max Heap

Definition: 一種具排序規則的 **Complete Binary Tree**，在符合 **Complete Binary Tree** 的規則前提下，必須同時滿足其堆順序性質(**Heap Property**)，樹中每個節點的值都必須大於或等於其子節點，因此根節點一定是整棵樹的最大值。

8. Min Heap

Definition: 一種具排序規則的 **Complete Binary Tree**，在符合 **Complete Binary Tree** 的規則前提下，必須同時滿足其堆順序性質(**Heap Property**)，樹中每個節點的值都必須小於或等於其子節點，因此根節點一定是整棵樹的最小值。

Section 2. Tree Family Hierarchy and Transformations

Task: Show how these structures are related (general $\rightarrow$ specialized). Use a simple diagram and explanations of what constraints are added at each step.

2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

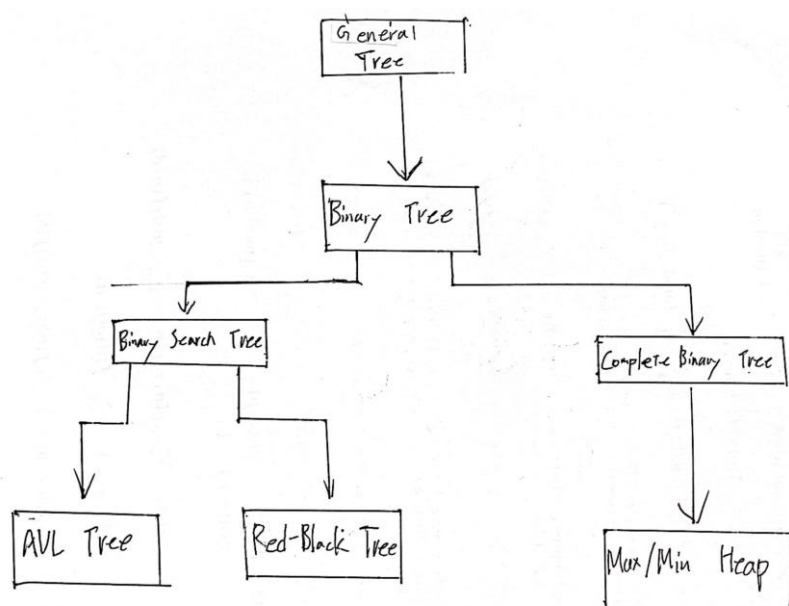
Suggested chain example (you may extend or adjust):

General Tree $\rightarrow$ Binary Tree $\rightarrow$ Complete Binary Tree

Binary Tree $\rightarrow$ Binary Search Tree $\rightarrow$ AVL / Red-Black

Binary Tree $\rightarrow$ Max Heap / Min Heap

Your Diagram:

**2.2 Explanation of Transformations**

Fill in what new property or constraint is added at each step.

From	To	New property / constraint added
General Tree	Binary Tree	每個節點最多兩個 Child

Binary Tree	Complete Binary Tree	除了最後一層之外，所有層都必須被完全填滿，而最後一層的節點必須由左向右連續排列
Binary Tree	Binary Search Tree	左子樹中的所有節點 $<$ 父節點 $<$ 右子樹中的所有節點 的排序規則
BST	AVL Tree	對於樹中任何一個節點，其左、右子樹的高度差不得超過 1
BST	Red-Black Tree	紅黑顏色規則 + 黑高度相同
Binary Tree	Max Heap	結構上必須是 Complete Binary Tree 還有 父節點 $\geq$ 子節點
Binary Tree	Min Heap	結構上必須是 Complete Binary Tree 還有 父節點 $\leq$ 子節點

Section 3. Tree Constructions Using Given Integers

Given integers (fixed for all parts):

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

3.1 Binary Tree

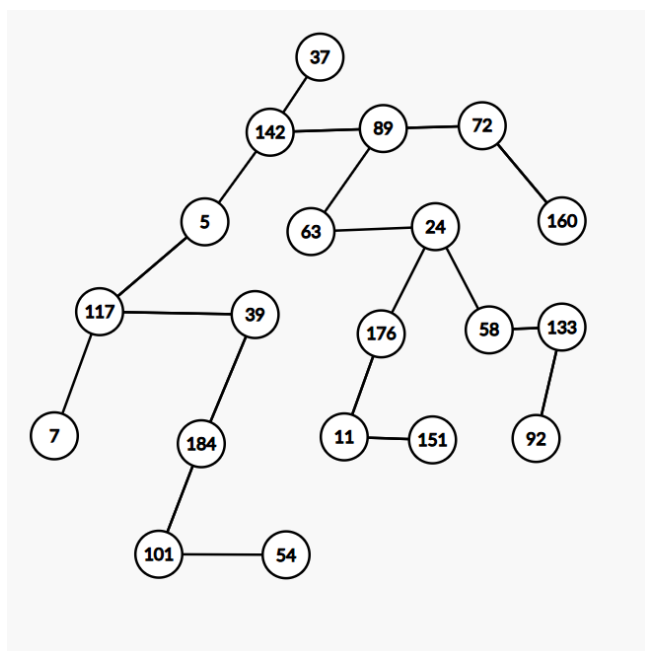
Tool name / URL:

https://csacademy.com/app/graph_editor/

Construction / insertion description:

從空樹開始，依照給定整數的順序依次插入節點。插入時隨機放置子節點數還小於 2 的節點下面。

Screenshot of Binary Tree (paste below):



3.2 Complete Binary Tree

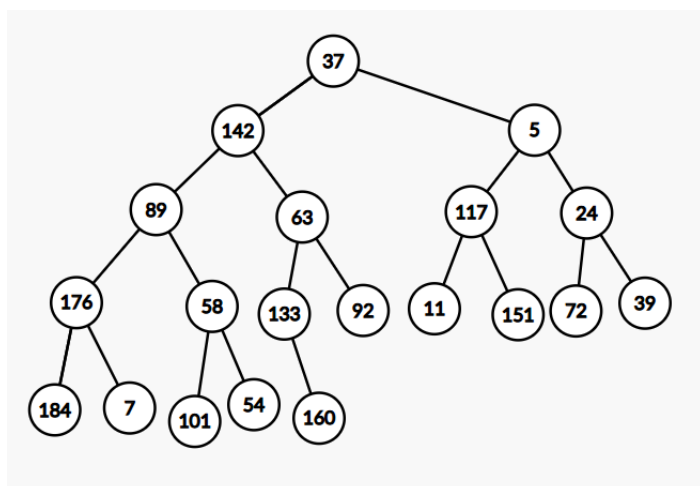
Tool name / URL:

https://csacademy.com/app/graph_editor/

Construction / insertion description:

從空樹開始，依照給定整數的順序逐一建立節點。插入時以樹的層級順序填入：先放根節點，再依序填滿同一層的節點，由左至右；當某一層填滿後，再往下一層繼續同樣的左到右插入，直到所有數字都放入，使樹保持為完全二元樹。

Screenshot of Complete Binary Tree (paste below):



3.3 Binary Search Tree (BST)

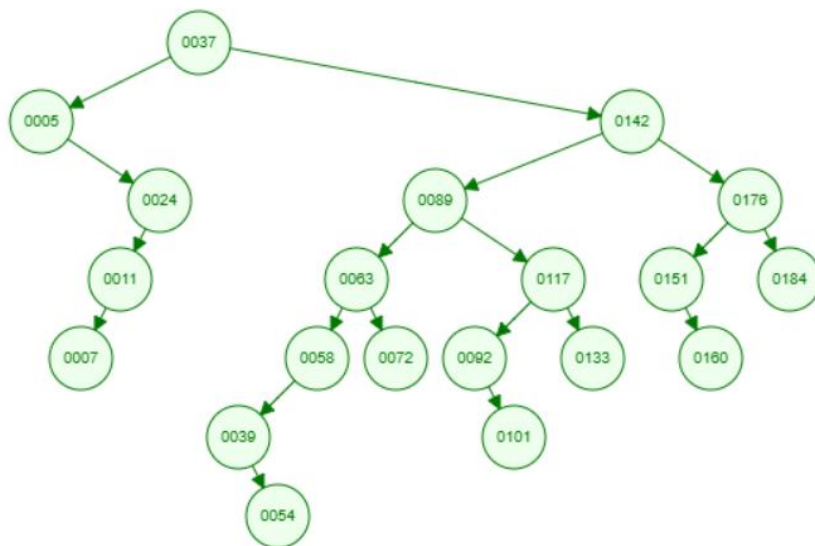
Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/BST.html>

Insertion rule (e.g., “insert in given order using BST rules”):

以 37 為根節點，之後依給定順序依次插入每一個整數。插入時遵守 BST 規則：新值小於目前節點就往左子樹走，大於目前節點就往右子樹走，直到走到空位置，把新節點掛上去。

Screenshot of BST (paste below):



3.4 AVL Tree

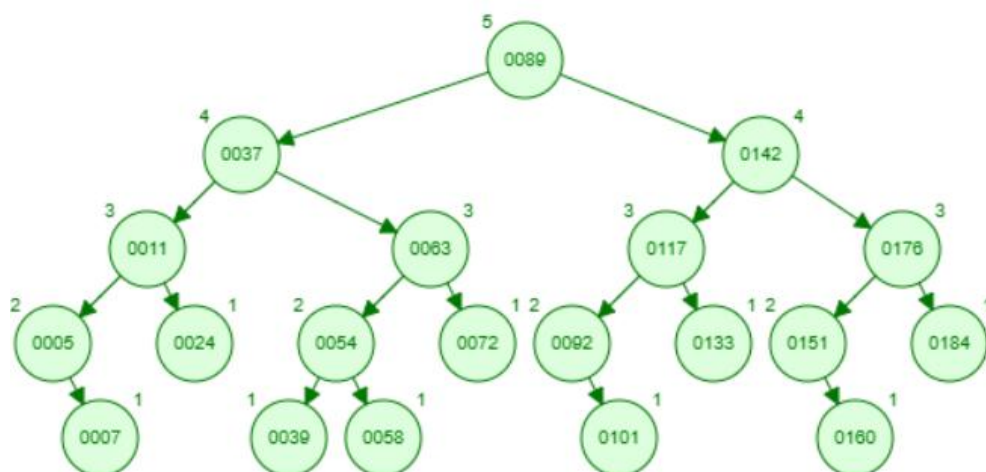
Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Insertion & balancing description:

從空樹開始，依照給定整數的順序，先用 BST 的規則插入新節點。在插入完成後，沿插入路徑往上更新每個節點的高度，並計算平衡因子 $BF = \text{height}(\text{left}) - \text{height}(\text{right})$ 。若某個祖先節點 z 出現 $|BF(z)| > 1$ 就表示失衡，並判斷是哪種類型的失衡並做對應的操作: $BF(z) > 1$ 且插入在 $z.\text{left}$ 的左邊為 LL (對 z 做右旋)， $BF(z) > 1$ 且插入在 $z.\text{left}$ 的右邊為 LR (先對 $z.\text{left}$ 左旋，再對 z 右旋)； $BF(z) < -1$ 且插入在 $z.\text{right}$ 的右邊為 RR (對 z 做左旋)， $BF(z) < -1$ 且插入在 $z.\text{right}$ 的左邊為 RL (先對 $z.\text{right}$ 右旋，再對 z 左旋)。旋轉後重新更新相關節點的高度與 BF ，如此在每次插入後都維持 AVL 的平衡性(高度為 $O(\log n)$)。

Screenshot of AVL Tree (paste below):



3.5 Red-Black Tree

Tool name / URL:

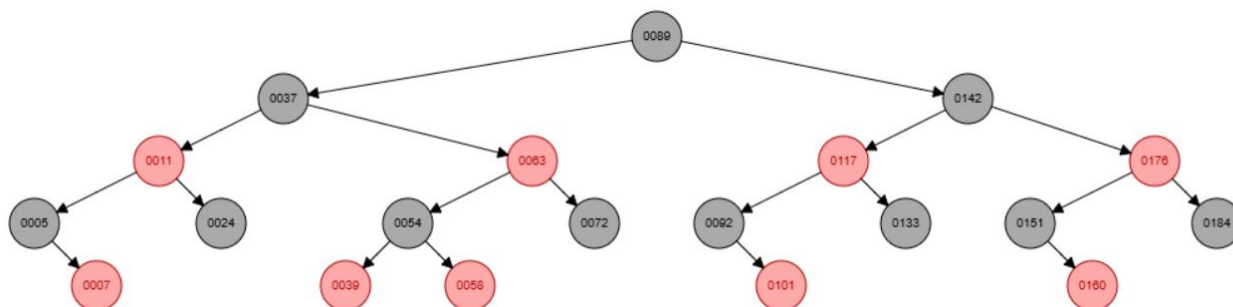
<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

Insertion & balancing description

依給定整數順序，先照 BST 規則插入新節點 z 插入時只看 **key** 大小決定走左或右，找到空位後接上，並將 z 著色為紅色。若 z 的父節點為黑色，所有紅黑性質仍成立，插入結束；若父節點為紅色，則違反「紅節點不能相鄰」的性質，此時以祖父 g 、父節點 p 、叔叔 u (p 的另一顆同輩) 來做修正：若叔叔 u 為紅色，就將 p 、 u 染成黑色， g 染成紅色，然後把 z 提升為 g 繼續往上檢查；若叔叔為黑色（或 NIL），則依 z 在父節點的內側或外側分成兩類：內側情況（類似 LR / RL 型）先對父節點做一次旋轉，讓 z 變成外側，再對祖父節點做一次旋轉，並在旋轉前後調整 p 、 g 的顏色（通常是新根變黑、另一個變紅），以消除「紅紅相連」又維持 **black-height** 一致。最後確保 **root** 被設為黑色，即是完成完整的插入與平衡。

Screenshot of Red-Black Tree (paste below):

這個 **visualization tool** 沒有把 **NIL** 顯現出來，但實際上是對的



3.6 Max Heap

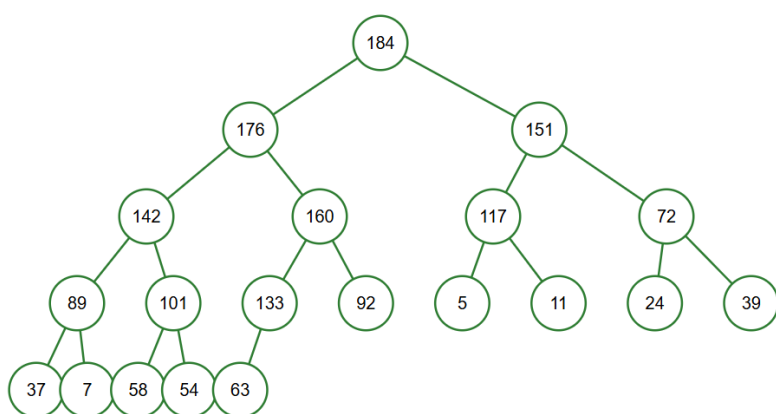
Tool name / URL:

https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Heap.html

Construction / heap-building description (e.g. heapify, insert-and-sift-up):

先將題目給的整數依順序放入一個陣列，並視其為一棵完全二元樹，接著用 **bottom-up** 的 **Heapify** 方式自底往上建 **heap**：從最後一個非葉節點開始，一路往 **index 0(root)** 做 **sift-down**。對每個節點 i ，比較它與左右子中較大的那個，若父節點已經 $\geq$ 最大子節點則停止；否則就與較大的子節點交換，並沿著那個子節點的方向繼續往下沉，直到該子樹滿足「父 $\geq$ 子」為止。所有非葉節點都處理完後，整個陣列就同時滿足完全二元樹結構與 **Max Heap** 性質，完成最大堆的建構。這個做法為 $O(N)$

Screenshot of Max Heap (paste below):



3.7 Min Heap

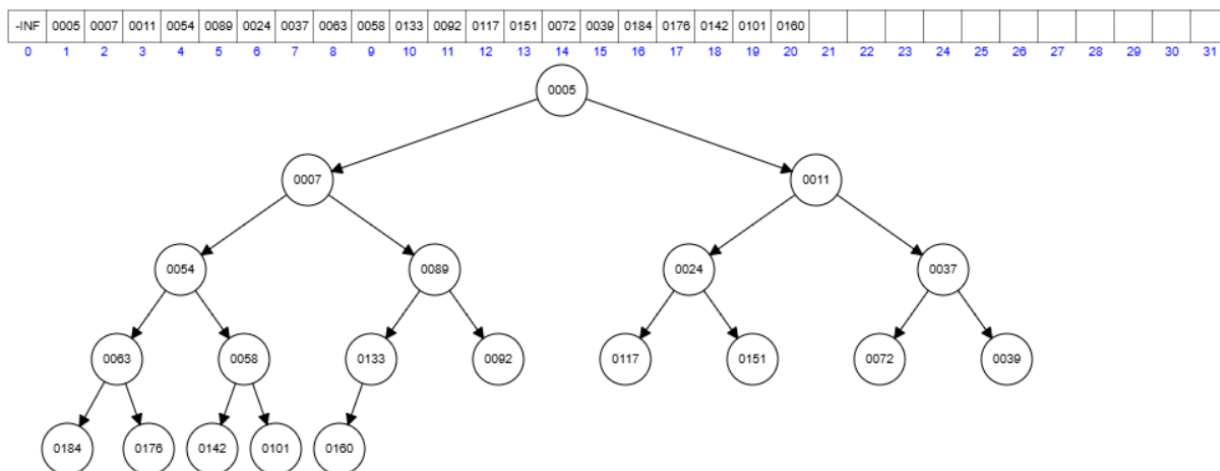
Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Construction / heap-building description:

先將題目給的整數依順序放入一個陣列，並視其為一棵完全二元樹，接著使用 **bottom-up** 的 **Heapify** 建堆方式：從最後一個非葉節點開始，一直到 **root(index 0)**，對每個節點做 **sift-down**。對節點 i ，比較它與左右子節點中較小的那個，若父節點已經 $\leq$ 較小子節點就代表滿足 **heap** 性質；否則就與較小的子節點交換，並沿著那個方向持續往下沉，直到該子樹恢復「父 $\leq$ 子」。當所有非葉節點都處理完後，就可以得到一棵同時滿足完全二元樹結構與 **Min Heap** 性質的最小堆。這個做法為 $O(N)$

Screenshot of Min Heap (paste below):



Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

Tree Type	Application Example (name / context)	Why this tree fits (properties that matter)
Binary Tree	表示運算表達式	因為每個運算子在運算樹中是一個節點，左右子樹代表運算元。因此可以透過前序、中序、後序遍歷可轉成不同表示，其中中序適合閱讀、後序適合計算
Complete Binary Tree	監控系統的時間序列資料處理	監控系統中的時間序列資料（如每秒錯誤數）以 Complete Binary Tree 儲存，可用區間樹 / Fenwick Tree 在 $O(\log n)$ 時間內做「最近一段時間錯誤總數」等區間查詢，適合高頻更新與查詢。
Binary Search Tree	字典 / 資料索引（如搜尋關鍵字、會員資料查詢）	利用 BST 「左小右大」的特性，可在平均 $O(\log n)$ 時間內完成搜尋、插入與刪除，效率高且適合需要頻繁查詢的系統。
AVL Tree	資料庫系統中需要大量搜尋與更新的情形	AVL Tree 會自動維持平衡，使高度保持在 $O(\log n)$ ，因此搜尋、插入、刪除都能穩定在 $O(\log n)$ 時間內完成，效能穩定不會退化。
Red-Black Tree	作業系統核心，需要高頻插入與刪除的情形	紅黑樹比 AVL 平衡條件較寬鬆，插入與刪除時需要的旋轉次數通常更少，維護成本低，在大量動態更新的情況下，能

		提供穩定的 $O(\log n)$ 效能與較佳實務表現
Max Heap	排程系統中的最高優先權任務佇列	以完全二元樹陣列儲存，能 $O(1)$ 取得最大值、 $O(\log n)$ 插入與刪除，實作簡單且效率高。
Min Heap	最短路徑演算法(Dijkstra)	利用 heap 可以在每次 RELAX 將時間成本壓到 $O(\log n)$

Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Among BST, AVL, and Red-Black trees, which one would you pick for:

Mostly search (few updates)? Why?

BST 表現最差所以先不考慮(不平衡)。

在盡可能多搜尋的情境下，AVL 更好，因為 AVL 是「嚴格平衡」的平衡樹，高度更接近 $\log n$ ，而 Red-black trees 則是相對寬鬆一點的 tree，他可以容忍 AVL 兩倍常數因子的高度差，因此在大量搜尋時，AVL 的路徑平均會比較短，所以 AVL 搜尋表現會比 Red-Black trees 好。

Frequent insertions and deletions? Why?

BST 表現最差所以先不考慮(不平衡)。

在大量插入與刪除的情況下，Red-Black Tree，因為 Red-Black Tree 的平衡條件比較寬鬆，所以插入與刪除時需要的旋轉次數通常比 AVL 少，在大量插入與刪除的場景下有較好的整體效能。

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

我覺得使用 hash table 最好，因為其平均查詢時間為 $O(1)$ ，而且因為資料是靜態的，適合預先設計良好的雜湊函數。

Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

Index	Date / Time	AI Service (ChatGPT, Gemini, etc.)	Your Full Prompt / Question
1		ChatGPT	我想了解資料結構相關的資訊，我想問 General Tree 是甚麼 他的明確定義是啥
2		ChatGPT	那 Binary Tree 是甚麼，它的結構與明確的定義是啥，跟 General Tree 又有何差別

3		ChatGPT	那請問 Complete Binary Tree 的明確定義又是甚麼，他跟前面的結構又有何差異
4		ChatGPT	那 Binary Search Tree (BST) 呢
5		ChatGPT	那 AVL Tree 呢
6		ChatGPT	那 Red-Black Tree 呢，他跟 AVL TREE 的差別又在哪
7		ChatGPT	那 max heap 呢
8		ChatGPT	那 min heap 呢 他跟 max heap 又差在哪
9		ChatGPT	請問 BST 的完整 Constructio 與插入規則是甚麼
10		ChatGPT	請你詳細的講解紅黑樹 Insertion & balancing 的規則
11		ChatGPT	能不能詳細講解 max heap 的 Insert-and-Sift-Up 規則
12		ChatGPT	那 Heapify 的詳細作法與規則呢
13		ChatGPT	請問 Binary Tree 有哪些適合應用的場景，以及為什麼適用
14		ChatGPT	那 Complete Binary Tree 呢 有哪些適合應用的場景，以及為什麼適用
15		ChatGPT	請問 Binary Search Tree 是如何讓搜尋變快的
16		ChatGPT	請問 AVL tree 有甚麼適合應用的場景
17		ChatGPT	請問紅黑樹有甚麼適合應用的場景他跟 AVL tree 適用的場景又差在哪邊
18		ChatGPT	請問 heap 有甚麼適合應用的情形
19		ChatGPT	我想問一下 AVL 跟紅黑數在盡可能多的搜尋情境下，哪種效益較高，為什麼？

You may extend this table as needed.